

Consignes générales

L'examen dure deux heures. Tous les documents sont autorisés. Tout matériel électronique doit être silencieux et rangé hors de vue. Merci de répondre dans l'ordre aux questions d'un même exercice. La clarté et la concision des réponses seront appréciées. Votre copie doit être cachetée et anonyme ; reportez uniquement votre numéro étudiant sur les éventuelles copies supplémentaires.

1 Analyse lexicale et syntaxique

Le fichier `parser.mly` ci-dessous contient un analyseur syntaxique pour un petit fragment de langage contenant uniquement des entiers et des types construits (sommes) avec filtrage de motif simple.

```
%{
  type id = string                                (* Identifiants *)
  type cname = string                             (* Constructeurs *)
  type pat = PVar of id | PCons of cname * pat list (* Motifs *)
  type exp = Var of id | Lit of int | Add of exp * exp (* Expressions *)
             | Cons of cname * exp list
             | Match of exp * (pat * exp) list
}%

%token EOF PLUS COMMA MATCH WITH END ARROW LPAREN RPAREN
%token<int> INT
%token<string> ID CNAME

%start<exp> start

%right PLUS

%%

parens(X): LPAREN x = X RPAREN { x }
cons(X): c = CNAME ps = parens(separated_nonempty_list(COMMA, X)) { (c, ps) }

start: exp EOF { $1 }

pat: x = ID { PVar x } | pc = cons(pat) { PCons pc }

branch: p = pat ARROW e = exp { (p, e) }

exp: x = ID { Var x } | k = INT { Lit k }
    | e1 = exp PLUS e2 = exp { Add (e1, e2) } | ec = cons(exp) { Cons ec }
    | MATCH e = exp WITH bs = branch* { Match (e, bs) }
```

- Quelle conséquence aurait la suppression de la directive `%right PLUS` du fichier ci-dessus ? Quelle(s) conséquence(s) aurait son remplacement par `%left PLUS` ? Illustrer la différence par une séquence de terminaux donnant lieu à des arbres de syntaxe distincts selon l'associativité de `PLUS`. Donner les arbres de syntaxe en question. Cette différence est-elle significative, et pourquoi ?
- La commande `menhir --explain parser.mly` détecte des conflits avancer/réduire tous liés à la construction de filtrage de motifs, dont celui qui figure à la figure 1. Illustrer ce dernier en donnant une séquence de terminaux ainsi que les deux arbres de syntaxes distincts qui résulteront de l'analyse de cette séquence selon que l'on aura choisi d'avancer ou de réduire.
- Proposer une modification minimale de la syntaxe qui résout tous les conflits avancer/réduire de la grammaire via l'utilisation d'un nouveau symbole terminal noté `END`.
- Proposer un analyseur lexical pour le petit langage obtenu. Votre proposition doit satisfaire les critères suivants.

```
** Conflict (shift/reduce) in state 13.
** Token involved: ID
** This state is reached from start after reading:

MATCH exp WITH pat ARROW MATCH exp WITH

** The derivations that appear below have the following common factor:

start
exp EOF
MATCH exp WITH list(branch)
      (?)

** In state 13, looking ahead at ID, reducing production
** list(branch) ->
** is permitted because of the following sub-derivation:

branch list(branch)
pat ARROW exp
      MATCH exp WITH list(branch)
      .

** In state 13, looking ahead at ID, shifting is permitted
** because of the following sub-derivation:

branch list(branch)
pat ARROW exp
      MATCH exp WITH list(branch)
      branch list(branch)
      pat ARROW exp
      . ID
```

FIGURE 1 – Un conflit expliqué par Menhir

```

module SSet : sig
  type t
  val empty : t
  val add : string -> t -> t
  val remove : string -> t -> t
  val union : t -> t -> t
  val mem : string -> t -> bool
  val fold : (string -> 'a -> 'a)
              -> t -> 'a -> 'a
end

module Dict : sig
  type 'a t
  val empty : 'a t
  val add : string -> 'a -> 'a t -> 'a t
  val find : string -> 'a t -> 'a
  val map : ('a -> 'b) -> 'a t -> 'b t
  val union : 'a t -> 'a t -> 'a t
  val of_list : (string * 'a) list
                -> 'a t
end

```

FIGURE 2 – Signatures des modules fournis

- Produire les terminaux **PLUS**, **COMMA**, **MATCH**, **WITH**, **END**, **ARROW**, **LPAREN**, **RPAREN** et **EOF**. Le terminal **PLUS** doit correspondre au symbole mathématique +, **COMMA** à la virgule, **ARROW** à la flèche →, et enfin **LPAREN** (resp. **RPAREN**) à la parenthèse ouvrante (resp. fermante). Les autres terminaux cités sont des mots-clefs du langage, écrits en minuscules.
 - Produire le terminal valué des identifiants **ID**. Un identifiant commence obligatoirement par une minuscule, suivie d'un nombre quelconque de minuscules, majuscules ou tirets de soulignement.
 - Produire le terminal valué des noms de constructeurs **CNAME**. Un nom de constructeur commence obligatoirement par une majuscule, suivie d'un nombre quelconque de minuscules, majuscules ou tirets de soulignement.
 - Produire le terminal valué des entiers relatifs **INT**. On rappelle qu'un entier relatif peut être négatif.
 - Ignorer les caractères blancs, c'est-à-dire les espaces, tabulations et sauts de ligne.
 - Lever l'exception prédéclarée **Syntax_error** en cas de caractère non reconnu.
- Votre analyseur lexical doit être aussi simple et lisible que possible tout en respectant ces contraintes.

2 Autour des fermetures

On considère deux mini-langages fonctionnels sans effets de bords. Chacun peut être vu comme un petit fragment d'OCaml. L'objectif de cet exercice sera d'écrire un compilateur du premier vers le second.

Infrastructure. Pour représenter et manipuler ces langages en OCaml, on utilise deux modules **SSet** et **Dict**, que l'on suppose donnés, permettant de représenter respectivement des ensembles de chaînes de caractères et des tables associatives indicées par des chaînes de caractères. Leurs signatures figurent à la figure 2.

Langage source. Le langage source est un fragment d'OCaml contenant uniquement :

- les constantes entières,
- l'addition de deux entiers,
- les fonctions anonymes à un argument,
- l'application d'une fonction à un argument.

Son arbre de syntaxe abstraite peut donc être représenté par le type OCaml `src_expr` décrit à la figure 3a. Toute expression du langage source qui s'évalue sans erreur résulte, selon la sémantique d'OCaml, en une valeur qui est soit un entier, soit une valeur fonctionnelle (implémentée par une fermeture). Ces valeurs peuvent donc être représentées par le type OCaml `src_val` défini dans la même figure.

Le langage cible. Le langage cible est un fragment *non typé* d'OCaml contenant uniquement :

- les constantes entières,
- l'addition de deux entiers,
- les définitions locales de *valeurs* (**let** `x = expr1` **in** `expr2`) mais pas de fonctions,
- les définitions *globales* de fonctions de la forme **let** `f (x1, ..., xn) = expr ;;`,
- l'application d'une fonction à un tuple d'expressions (`f 4, g (1, 1+1)`, etc.),

```

type src_expr =
  | SVar of string          (* Occurrence de variable *)
  | SConst of int           (* Constante littérale entières *)
  | SAdd of src_expr * src_expr (* Addition entière *)
  | SFun of string * src_expr (* Fonction anonyme (unaire) *)
  | SApply of src_expr * src_expr (* Application de fonction (unaire) *)
type src_val = SInt of int | SClosure of src_clo
and src_clo = { env : src_val Dict.t; argument : string; body : src_expr; }

```

(a) Syntaxe abstraite du langage source

```

type tgt_expr =
  | TVar of string          (* Occurrence de variable *)
  | TConst of int           (* Constante littérale entière *)
  | TAdd of tgt_expr * tgt_expr (* Addition entière *)
  | TLet of string * tgt_expr * tgt_expr (* Définition locale *)
  | TRecord of tgt_expr Dict.t (* Construction d'enregistrement *)
  | TField of tgt_expr * string (* Accès à un champ *)
  | TApply of tgt_expr * tgt_expr list (* Application de fonction (n-aire) *)
type tgt_function = { arguments : string list; body : tgt_expr; }
type tgt_program = tgt_function Dict.t * tgt_expr
type tgt_value = TInt of int | TFunVal of string
                | TRecordVal of tgt_value Dict.t

```

(b) Syntaxe abstraite du langage cible

FIGURE 3 – Syntaxe abstraite des langages considérés

- les enregistrements (`{ a = 1 ; b = 1+1 }`) (remarque : puisqu'il s'agit d'un fragment non typé, les types d'enregistrement n'ont pas à être déclarés à l'avance),
- l'accès à un champ d'enregistrement (`expr.champ`).

Notez bien qu'il n'y a pas de fonctions anonymes ni de définitions de fonctions locales dans le langage cible.

Un programme du langage cible est constitué d'une liste de définition de fonctions, suivie d'une expression qui constitue le code principal du programme, auquel on peut penser comme au corps de la fonction `main` en langage C. Il peut donc être représenté par une valeur du type `tgt_program` défini à la figure 3b.

L'évaluation d'une expression du langage cible produit soit une erreur, soit un entier, soit un enregistrement, soit une fonction *parmi celles définies à toplevel*. Le résultat de l'évaluation peut donc être représenté par le type OCaml `tgt_value` défini dans la même figure.

2.1 Évaluation

- Définir une expression OCaml `tgt_prog_1` de type `tgt_program` représentant l'arbre de syntaxe abstraite du programme suivant du langage cible.

```

let f (x, y) = { a = x.a ; b = x.b + y } ;;
f { a = 1 ; b = 20 } 300

```

- Vers quelle valeur s'évalue le programme de la question précédente? Quelle valeur de type `tgt_value` la représente?
- Définir une expression OCaml `src_expr_1` de type `src_expr` qui représente l'expression suivante du langage source.

```

fun x -> fun y -> x + y

```

- Donner une expression OCaml de type `src_val` qui représente le résultat de l'évaluation de l'expression de la question précédente.

```

let pure e = (Dict.empty, e)
let merge f (d1, e1) (d2, e2) = (Dict.union d1 d2, f e1 e2)

let rec compile : src_expr -> tgt_program =
  function SVar x -> pure (TVar x) | SConst k -> pure (TConst k)
  | SAdd (e1, e2) ->
    merge (fun e1 e2 -> TAdd (e1, e2)) (compile e1) (compile e2)
  | SFun (argument, body) -> handle_fun argument body
  | SApply (e1, e2) -> merge handle_app (compile e1) (compile e2)

and handle_app tgt_fun tgt_arg = (* ... *)

and handle_fun argument src_body = (* ... *)

```

FIGURE 4 – Explicitation des fermetures à compléter

e. Définir une expression OCaml `src_expr_2` de type `src_expr` qui représente l'expression suivante du langage source.

```
(fun add -> add 1 2) (fun x -> fun y -> x + y)
```

f. Écrire une fonction récursive

```
eval_src : src_val Dict.t -> src_expr -> src_val
```

telle que `eval_src env e` évalue l'expression `e` en supposant que les valeurs de ses variables libres sont données par l'environnement `env`. En cas d'erreur, votre fonction pourra lever une exception de votre choix.

2.2 Compilation

Nous allons maintenant nous intéresser à la traduction du langage source vers le langage cible, qui consiste essentiellement en l'étape d'*explicitation des fermetures* décrite en cours. Il s'agit donc de représenter les fermetures comme des structures de données du langage cible.

Pour toutes les questions suivantes :

- dans le code généré, les fermetures devront être représentées par des enregistrements, avec un codage de votre choix mais fixé ;
- on suppose que dans le langage source, tous les noms de variables commencent par une lettre minuscule, et qu'il n'y a pas de restriction de ce genre dans le langage cible.

Infrastructure. On supposera donnée une fonction `fresh_name : unit -> string` dont chaque appel renvoie la chaîne de caractère `"_ " ^ suffix`, où `suffix` est une chaîne de caractères alphanumériques qui change à chaque appel, et telle qu'un même suffixe n'est jamais renvoyé deux fois durant une même exécution de votre programme. Notez que la chaîne de caractères renvoyée par `fresh_name ()` commence toujours par un tiret de soulignement, et ne peut donc jamais apparaître comme identifiant du langage source.

Exemples. L'expression `fun x -> x + 1` peut être traduite vers le programme

```

let _fun_1 (_this, x) = x + 1;; (* définition de fonction toplevel *)
{ _function = _fun_1 }

```

tandis que l'expression `f 12` peut être traduite vers le programme

```

(* pas de définitions de fonctions toplevel *)
let _closure_1 = f in _closure_1._function(_closure_1, 12)

```

Questions. L'objectif de cette partie finale est de compléter la figure 4, qui contient une fonction OCaml

```
compile : src_expr -> tgt_program
```

traduisant les expressions du langage source vers des programmes du langage cible en réalisant l'explicitation des fermetures. On commence par exécuter la fonction sur des exemples pour comprendre son comportement attendu, avant de compléter son code.

- a. Traduire vers un programme du langage cible (en effectuant donc l'explicitation des fermetures) les expressions suivantes du langage source. On demande le résultat sous forme de code source écrit en imitant la syntaxe d'OCaml, comme dans les exemples qui précèdent, et non pas sous la forme d'une expression de type `tgt_program`.

1. **fun** x -> **fun** y -> x + y

2. **fun** f -> f 42

3. (**fun** add -> add 1 2) (**fun** x -> **fun** y -> x + y)

- b. Définir une fonction OCaml `src_free_variables : src_expr -> SSet.t` qui calcule l'ensemble des variables libres d'une expression du langage source.
- c. Définir la fonction `handle_app` manquant à la figure 4.
- d. Définir la fonction `handle_fun` manquant à la figure 4.